

SensOS

Starter Guide

New User Edition

From installation to your first engineering experiment.

Simple by default. Composable by design.

Product path



Contents

Welcome	2
1 What SensOS is	2
2 Install (primary path)	3
2.1 Supported platform	3
2.2 Download	3
2.3 Install	3
2.4 If <code>uv</code> is missing	4
2.5 If <code>./install.sh</code> is not in your checkout	4
3 Launch	4
3.1 Optional: live smoke (after you start inference)	5
4 Start Building	5
5 Open the Workspace	5
6 Create your first Goal	6
6.1 Steps	6
6.2 Allowed connections	6
6.3 Rejected connections	6
6.4 Branch and Join	6
6.5 Execution policies on a Milestone	7
7 Assemble a Harness	7
7.1 Steps	7
8 Select a Run	7
9 Inspect a Trajectory	8
9.1 Steps	8
10 Meet HANKO	8
11 Chat command cheat sheet	8
12 First successful session checklist	9
13 What to do next	9
14 Advanced: Studio Workspace	10
15 What this guide deliberately skips	10
16 Troubleshooting	10
17 Repository note	11

Welcome

Welcome to SensOS. This guide gets you from zero to building today.

The product path is:



You will:

1. Install SensOS with the **official Linux installer**
2. Launch the SensOS CLI and confirm the environment
3. Start Building: create a Goal, assemble a Harness, select a Run, inspect a Trajectory, and meet HANKO

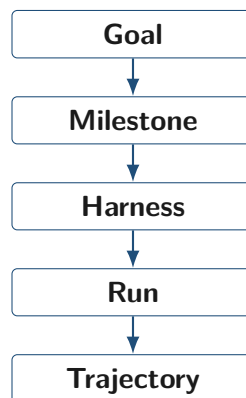
IMPORTANT

You do **not** need to understand kernel internals, mathematics, or private APIs to complete this guide.

1 What SensOS is

SensOS is an observation-centered product surface for installing a runtime, designing goals, assembling agent loops, inspecting recorded behavior, and recording human authority decisions.

Keep this picture in mind:



Product vocabulary

Term	Meaning
Goal	What you want the system to achieve (Purpose → Milestones → Final Goal)
Milestone	An intermediate checkpoint on the path to the Final Goal
Harness	The Observe / Reason / Act graph that executes work
Run	One execution instance
Trajectory	The ordered evidence recorded for a Run
Gauge	An instrument attached to an observation point on the Harness
HANKO	A Milestone policy that requires explicit human authority

Product terms stay in English as written above. Short explanations appear only on first use.

2 Install (primary path)

SensOS ships a **Linux installer**: `install.sh`.

That installer is the **primary recommended installation path**. Do not begin with a manual dependency tree unless the installer itself tells you to install a missing prerequisite.

2.1 Supported platform

SUPPORTED

Linux — Supported (fresh install verified on Ubuntu 24.04).

WARNING / LIMITATION

macOS / Apple Silicon — Not supported by this installer (installer exits if `uname -s` is not `Linux`).

Windows — Not supported by this installer.

Platform	Installer support
Linux	Supported (fresh install verified on Ubuntu 24.04)
macOS / Apple Silicon	Not supported by this installer (installer exits if <code>uname -s</code> is not <code>Linux</code>)
Windows	Not supported by this installer

Prerequisites checked by the installer:

- **Python** $\geq$ **3.12**
- **uv** on `PATH` (if missing, the installer prints the official `uv` install command and exits)

2.2 Download

```
git clone https://github.com/GemminAI/sensos.git
cd sensos
```

2.3 Install

```
./install.sh
```

What `./install.sh` installs (and only this):

- SensOS Runtime (`sensos` package)
- Semantic Annotator (editable dependency)
- vLLM (Linux inference backend package)
- Required Python dependencies via `uv sync / uv pip install`

What it does **not** do:

- Start a vLLM server
- Download or pin a model
- Install desktop apps, Docker stacks, or Studio frontends

Optional dry run (no changes):

```
./install.sh --dry-run
```

Help:

```
./install.sh --help
```

2.4 If `uv` is missing

The installer fails closed and prints the prerequisite command. Install `uv`, then re-run `./install.sh`:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

2.5 If `./install.sh` is not in your checkout

The official user-facing flow is `./install.sh` inside the SensOS package directory (the tree that contains `pyproject.toml` with `name = "sensos"`).

If your clone of [GemminAI/sensos](#) does not yet contain `install.sh`, the installer migration into that public tree is incomplete. Use the verified SensOS package that **does** ship `install.sh` — do **not** invent a substitute manual install of vLLM, torch, or annotator paths.

Verified invocation (this machine):

- `./install.sh --help` → Usage printed, exit 0
- `./install.sh --dry-run` on non-Linux →
FAIL: this installer targets Linux, exit 1 (expected)
- Fresh Linux install path verified on Ubuntu 24.04 Docker per the SensOS package README



3 Launch

After a successful install, launch from the same SensOS package directory:

```
uv run sensos version
uv run sensos doctor
```

Command	Purpose
<code>uv run sensos version</code>	Package version and source commit
<code>uv run sensos doctor</code>	Environment checks (OS, Python, uv, SensOS, Semantic Annotator, vLLM, GPU)
<code>uv run sensos smoke</code>	Minimal live RuntimeBridge → vLLM → LLMAnnotator check

`doctor` may report GPU FAIL on machines without CUDA. That is an honest environment report, not an installer crash.

3.1 Optional: live smoke (after you start inference)

The installer does not start a model server. When you are ready:

```
uv run vllm serve <model> --port 8000 --dtype auto
export RUNTIME_BRIDGE_URL=http://localhost:8000
export SENSOS_MODEL_ID=<model>
uv run sensos smoke
```

Choose `<model>` yourself. There is no default model in the installer.

You are now installed and launched. Next: **Start Building**.

4 Start Building

With SensOS installed, build in product vocabulary:



The sections below use the SensOS Studio Workspace (Goal / Harness / Trajectory panels). Studio is the interactive building surface for those objects. It is separate from `install.sh`; see **Advanced: Studio Workspace** if you need to open it locally.

5 Open the Workspace

When the Studio loads, you are in one Workspace. Goal, Harness, Run, and Trajectory switch in place — you do not jump between separate apps.

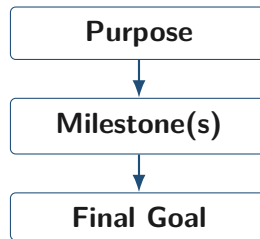
Typical panels:

- **Goal** — plan Purpose, Milestones, and Final Goal
- **Harness** — design Observe / Reason / Act
- **Trajectory** — replay recorded evidence
- **Chat** — deterministic commands (no LLM required)
- **Inspector** — details for the selected node or event

If a panel is hidden, use the Workspace layout controls to show Goal, Harness, or Trajectory.

6 Create your first Goal

A Goal project always follows this shape:



6.1 Steps

1. Click **New Project**, or type `new project` in Chat.
2. You start with **Purpose**, **Milestone 1**, and **Final Goal**. No edges yet.
3. Connect **Purpose** → **Milestone 1**.
4. Connect **Milestone 1** → **Final Goal**.
5. Optional: add another Milestone from the palette, then connect it.

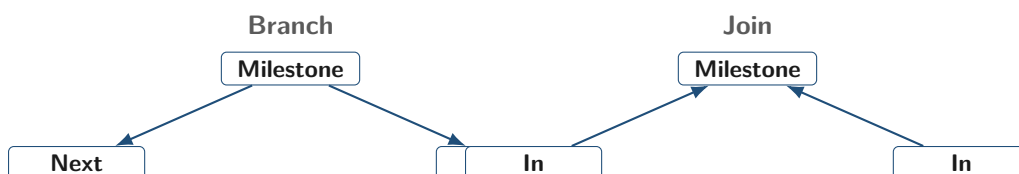
6.2 Allowed connections

- Purpose → Milestone
- Purpose → Final Goal (minimal project, no Milestone)
- Milestone → Milestone
- Milestone → Final Goal

6.3 Rejected connections

- Final Goal → anything
- Anything → Purpose
- Cycles and self-loops

6.4 Branch and Join



- **Branch** = one Milestone connects to more than one next step
- **Join** = more than one Milestone connects into one next step

Branch and Join are ordinary connection patterns. They are not special node types.

6.5 Execution policies on a Milestone

Select a Milestone and choose a policy:

Policy	Behavior
AUTO	Proceed without stopping for a human
NOTIFY	Notify when the Milestone is reached
PAUSE	Pause at the Milestone
HANKO	Require explicit human authority (see below)

7 Assemble a Harness

A **Harness** is the runtime graph that observes the world, reasons, and acts.

The starter Harness uses three node kinds:



7.1 Steps

1. Switch the Workspace to **Harness** (or type `show harness` in Chat).
2. Inspect the demo graph: Observe, Reason, Act, plus ports and instruments.
3. Add or rearrange nodes from the palette as needed.
4. Attach a **Gauge** to an ObservationPort when you want a visible instrument on that edge.

Useful Chat commands:

```
show harness
add gauge
add risk gauge
```

You are designing the loop that will produce Runs and Trajectories. You are not configuring kernel internals.

8 Select a Run

A **Run** is one execution of the Harness.

In Studio mock mode, a demo Run is already available:

1. Open the Run list in the Workspace dock or Runtime area.
2. Select `run-demo` (or the Run shown in your build).
3. Confirm the Inspector shows Run identity and status.

Live Runs appear when a SensOS API is connected. Until then, the public demo is enough to learn Trajectory inspection.

9 Inspect a Trajectory

A **Trajectory** is the ordered evidence for a Run — what happened, in sequence.

9.1 Steps

1. Type `show trajectory` in Chat, or open the Trajectory panel.
2. Select the Run if it is not selected.
3. Walk the event list / timeline.
4. Open an event in the Inspector to see observation, control, and provenance fields.

You are reading **what was recorded**, not editing the past.

Demo Trajectories are labeled as public demo evidence. Hash-chain verification and live provenance require a connected SensOS API.

10 Meet HANKO

NOTE

HANKO is the human-authority boundary on a Milestone.
HANKO requires explicit human authority.

When a Milestone policy is `HANKO`:

- The Workspace opens an Approval Surface for that Milestone.
- Local AI / Chat commands cannot change or skip HANKO.
- Human intents include **ACCEPT**, **REJECT**, **OVERRIDE**, and **ESCALATE**.

Human intents

ACCEPT

REJECT

OVERRIDE

ESCALATE

In the public Studio, those intents stay local until a SensOS API is connected. The important idea is product-level: **promotion authority belongs to a human**, not to an automated renderer.

Try it:

1. Select a Milestone.
2. Set policy to `HANKO` (or Chat: `set policy hanko`).
3. Review the Approval Surface.
4. Confirm Chat cannot clear HANKO for you.

11 Chat command cheat sheet

Chat is a deterministic command surface. Useful starters:

`help`

```

new project
add milestone
duplicate milestone
connect purpose milestone
connect milestone final
set policy auto|notify|pause|hanko
set retry 3
set join all|first|threshold
show harness
show trajectory
add gauge
inspect hanko

```

Type `help` anytime for the current command list.

12 First successful session checklist

Install → Launch

- ☐ `./install.sh` completed on Linux (or dry-run confirmed on your machine)
- ☐ `uv run sensos version` prints a version
- ☐ `uv run sensos doctor` runs (GPU FAIL without CUDA is acceptable)

Start Building

- ☐ A Goal project exists with Purpose → Milestone → Final Goal
- ☐ You opened the Harness and saw Observe / Reason / Act
- ☐ You selected a Run
- ☐ You opened a Trajectory and inspected at least one event
- ☐ You set one Milestone to `HANKO` and saw the Approval Surface

If Install → Launch works, you have a real SensOS environment. If the Start Building items work, you understand the public product loop.

13 What to do next

Next step	Where
Deeper product walkthrough	examples/goal-planning.md (sensos-ux)
Architecture and design principles	docs/SENSOS_TECHNICAL_GUIDE.md (sensos-ux)
Open Standards & public RFCs	https://sensos.org · GemminAI/sensos-docs
Live Studio / API	Advanced section below

14 Advanced: Studio Workspace

ADVANCED

Use this only if you need the interactive Goal / Harness / Trajectory UI. It is **not** a replacement for `./install.sh`.

Requirements: Node.js 20+ (22 recommended), npm, a modern browser.

```
git clone https://github.com/GemminAI/sensos-ux.git
cd sensos-ux
npm install
npm run dev
```

Open <http://localhost:5173>.

Default API mode is **mock** — Goal Planning, Harness editing, and a demo Trajectory work without a backend.

Optional live API (after SensOS runtime / bridge is available):

```
VITE_SENSOS_API=live npm run dev
```

Vite proxies `/api` to `VITE_API_PROXY_TARGET` (default `http://localhost:8000`). The Studio talks only through the public SensOS API adapter. It never imports kernel source.

15 What this guide deliberately skips

This Starter Guide does **not** teach:

- NVS-Kernel proprietary algorithms or calibration
- Internal module inventories or experiment IDs
- Partner wire protocols or trade-secret execution strategies
- How to rewrite history or promote experience without human authority
- Manual recreation of what `install.sh` already installs

Those topics belong in later technical, partner, or private materials. Learn Install → Launch → Start Building first.

16 Troubleshooting

Symptom	What to try
FAIL: <code>this installer targets Linux</code>	Expected on macOS/Windows — use a Linux host or Ubuntu 24.04 environment
FAIL: <code>uv not found</code>	Install <code>uv</code> with the command printed by the installer, then re-run <code>./install.sh</code>
FAIL: <code>Python >=3.12 required</code>	Install Python 3.12+ and re-run

Symptom	What to try
<code>./install.sh</code> missing	You are not in the SensOS package tree that ships the installer — do not invent a manual install
<code>doctor</code> GPU FAIL	Expected without CUDA — other PASS lines still matter
<code>smoke</code> cannot reach bridge	Start <code>vllm serve</code> yourself; set <code>RUNTIME_BRIDGE_URL</code> and <code>SENSOS_MODEL_ID</code>
Blank Workspace panels	Switch layout preset to SensOS and enable Goal / Harness / Trajectory
No Trajectory events	Select a Run first (<code>run-demo</code> in Studio mock mode)
Cannot connect an edge	Check Allowed / Rejected connection rules above
Chat cannot clear HANKO	Expected — HANKO requires human authority

17 Repository note

Installer and CLI live in the **SensOS package** (`install.sh`, `uv run sensos ...`), intended for [GemminAI/sensos](#).

This Starter Guide Markdown source is published from the public UX docs tree:

<https://github.com/GemminAI/sensos-ux>

Related:

- [GemminAI/sensos-docs](#) — Open Standards and public RFCs
- <https://sensos.org>